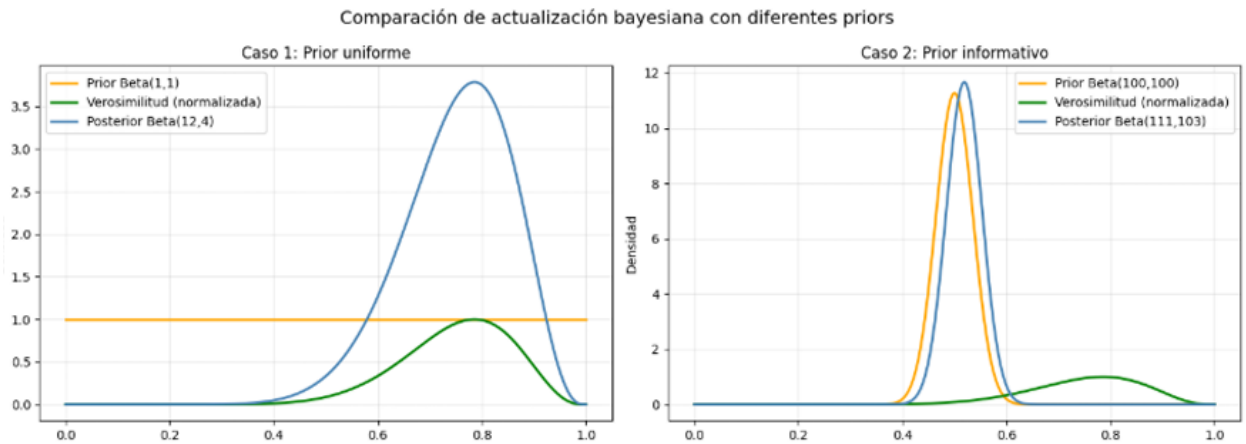
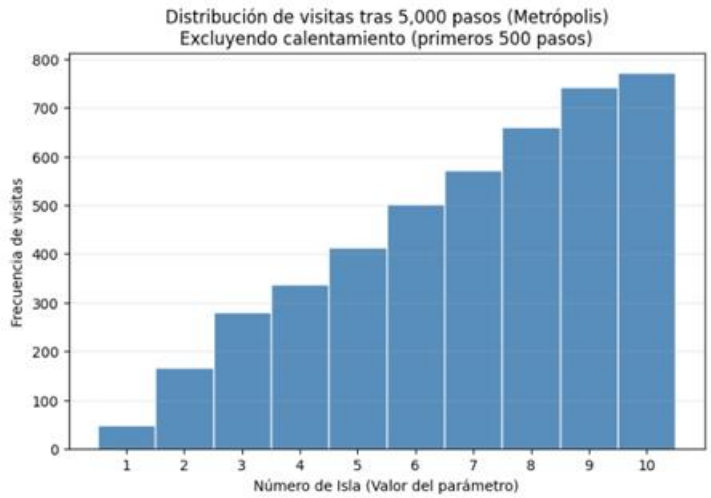
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta

# Caso 1: Prior uniforme Beta(1,1)
N1, z1 = 14, 11
a1, b1 = 1, 1
post1_a, post1_b = z1 + a1, N1 - z1 + b1

# Caso 2: Prior informativo Beta(100,100)
N2, z2 = 14, 11
a2, b2 = 100, 100
post2_a, post2_b = z2 + a2, N2 - z2 + b2

# Rango de valores para theta
theta = np.linspace(0, 1, 500)

# Calcular densidades
prior1 = beta.pdf(theta, a1, b1)
likelihood1 = theta**z1 * (1-theta)**(N1-z1)
likelihood1 /= np.max(likelihood1)  # normalizar para graficar
posterior1 = beta.pdf(theta, post1_a, post1_b)

prior2 = beta.pdf(theta, a2, b2)
likelihood2 = theta**z2 * (1-theta)**(N2-z2)
likelihood2 /= np.max(likelihood2)
posterior2 = beta.pdf(theta, post2_a, post2_b)

# Crear figura con dos subplots
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Subplot 1: Prior uniforme
axes[0].plot(theta, prior1, color="orange", lw=2, label=f"Prior Beta({a1},{b1})")
axes[0].plot(theta, likelihood1, color="green", lw=2, label="Verosimilitud (normalizada)")
axes[0].plot(theta, posterior1, color="steelblue", lw=2, label=f"Posterior Beta({post1_a},{post1_b})")
axes[0].set_title("Caso 1: Prior uniforme")
```

```python
axes[0].grid(alpha=0.3)

# Subplot 2: Prior informativo
axes[1].plot(theta, prior2, color="orange", lw=2, label=f"Prior Beta({a2},{b2})")
axes[1].plot(theta, likelihood2, color="green", lw=2, label="Verosimilitud (normalizada)")
axes[1].plot(theta, posterior2, color="steelblue", lw=2, label=f"Posterior Beta({post2_a},{post2_b})")
axes[1].set_title("Caso 2: Prior informativo")
axes[1].set_xlabel(r"")
axes[1].set_ylabel("Densidad")
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.suptitle("Comparación de actualización bayesiana con diferentes priors", fontsize=14)
plt.tight_layout()
plt.show()

# Imprimir medias
print("Caso 1:")
print(f"Media prior: {a1/(a1+b1):.3f}, Media posterior: {post1_a/(post1_a+post1_b):.3f}, Proporción observada: {z1/N1:
print("Caso 2:")
print(f"Media prior: {a2/(a2+b2):.3f}, Media posterior: {post2_a/(post2_a+post2_b):.3f}, Proporción observada: {z2/N2:
```




```
so 1:
dia prior: 0.500, Media posterior: 0.750, Proporción observada: 0.786
so 2:
```

```python
# 1. Definimos las probabilidades objetivo (P) para 10 "islas"
# Creamos un arreglo donde el índice coincide con la isla (1 a 10).
# El índice 0 lo rellenamos con un 0 porque no hay isla cero.
P_objetivo = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# 2. Función completa del Algoritmo de Metrópolis
def metropolis_walk(pasos, P, inicio=1):
    n_estados = len(P) - 1 # 10 estados válidos
    posicion_actual = inicio
    registro = np.zeros(pasos, dtype=int)

    for i in range(pasos):
        registro[i] = posicion_actual

        # Proponer un movimiento: lanzar moneda (-1 ir izquierda, +1 ir derecha)
        propuesta = posicion_actual + np.random.choice([-1, 1])

        # Verificar que la propuesta no se salga del mapa (islas 1 a 10)
        if 1 <= propuesta <= n_estados:

            # Regla de Metrópolis: Probabilidad de aceptar moverse
            p_mover = min(P[propuesta] / P[posicion_actual], 1.0)

            # Lanzamos un dado uniforme de 0 a 1. Si es menor a p_mover, nos movemos.
            if np.random.rand() < p_mover:
                posicion_actual = propuesta

    return registro

# 3. Ejecutamos la simulación
np.random.seed(123) # Para que la aleatoriedad sea reproducible
caminata = metropolis_walk(pasos=5000, P=P_objetivo, inicio=5)

# 4. Graficamos los resultados
# Excluimos los primeros 500 pasos (burn-in o "calentamiento")
datos_caminata = caminata[500:]

plt.figure(figsize=(8, 5))
# Dibujamos un histograma centrando las barras en los números enteros (islas)
plt.hist(datos_caminata, bins=np.arange(0.5, 11.5, 1),
         color='steelblue', edgecolor='white', alpha=0.9)
```

```python
# 4. Graficamos los resultados
# Excluimos los primeros 500 pasos (burn-in o "calentamiento")
datos_caminata = caminata[500:]

plt.figure(figsize=(8, 5))
# Dibujamos un histograma centrando las barras en los números enteros (islas)
plt.hist(datos_caminata, bins=np.arange(0.5, 11.5, 1),
         color='steelblue', edgecolor='white', alpha=0.9)

plt.xticks(range(1, 11))
plt.title('Distribución de visitas tras 5,000 pasos (Metrópolis)\nExcluyendo calentamiento (primeros 500 pasos)')
plt.xlabel('Número de Isla (Valor del parámetro)')
plt.ylabel('Frecuencia de visitas')
plt.grid(axis='y', alpha=0.3)
```